

Exercice 4 — Écosystème Spring : IoC, Spring Data JPA, Spring Security

Thème : *injection de dépendances, repository, API REST, authentification*

Contexte

On souhaite exposer, avec Spring Boot, une API REST permettant de consulter la liste des cours définis à l'exercice 3, et sécuriser cette API.

Objectifs

- Utiliser l'injection de dépendances de Spring (@Autowired / injection par constructeur).
- Créer un repository Spring Data JPA sans écrire d'implémentation.
- Exposer un endpoint REST avec Spring MVC.
- Protéger l'endpoint avec une authentification simple.

Travail demandé

- 1 Créez une interface CoursRepository qui étend JpaRepository<Cours, Long>, en ajoutant une méthode de recherche par intitulé, sans écrire de code d'implémentation.
- 2 Créez un service CoursService qui reçoit CoursRepository par **injection de dépendances via le constructeur** (et non un attribut public).
- 3 Créez un contrôleur REST CoursController annoté @RestController, exposant un endpoint GET /api/cours qui retourne la liste des cours au format JSON.
- 4 Configurez **Spring Security** pour que ce endpoint ne soit accessible qu'aux utilisateurs authentifiés (rôle USER minimum), en vous appuyant sur le principe du **RBAC**.
- 5 Expliquez en quelques lignes en quoi l'injection de dépendances par constructeur (plutôt que par attribut) facilite l'écriture de tests unitaires pour CoursService.

Critères d'évaluation

- Le repository ne contient aucune implémentation manuelle.
- L'injection de dépendances se fait par constructeur.
- L'endpoint est effectivement protégé par une règle d'autorisation.